FLOWER CLASSIFICATION WITH CUSTOM CNNS AND DAFFODIL SEGMENTATION VIA TRANSFER LEARNING

Samiksha Vanga
School of Computer Science, University of Nottingham, UK

ABSTRACT

This paper explores the application of deep learning techniques with two distinct tasks: the use of Convolutional Neural Networks (CNNs) for classification and DeepLabv3+ with ResNet-50 backbone for segmentation. For classification the CNN was trained on the Oxford 17 Flower Dataset, achieving an average accuracy of 73.90% demonstrating that a lightweight architecture can generalise well without transfer learning. For segmentation, a separate dataset of daffodil images was used to achieve an average accuracy of 92.83%. These findings support the potential of custom CNNs for constrained classification tasks and the suitability of pre-trained models for accurate semantic segmentation, even with limited data.

1. INTRODUCTION

Automated visual analysis of botanical images is a growing interest in computer vision due to its many applications in agriculture, biodiversity monitoring and education. Flowers, in particular, present a challenge within this field due to their variations in colour, shape, and texture. These challenges make the need for accurate classification and segmentation models of flower images necessary to ensure a high performance across environments.

A commonly cited work in flower classification is by Nilsback and Nisserman [1], who developed the Oxford 17 flower dataset used in this study. The paper details their approach which uses SVM-based classifiers on handcrafted features. While this method is effective to some extent, it often fails to generalise across diverse real-world conditions such as changes in background and lighting.

To address these limitations, pretrained CNNs such as VGGNet and ResNet were developed which have been repurposed for fine grained classification and significantly improve accuracy [2] [3]. However pre-trained networks rely on transfer learning which can introduce redundancy or inefficiency in scenarios where lightweight, domain-specific models are preferred.

Considering these reasons, training a CNN from scratch has been explored which offers more control over the design architecture and better adaptation to specific image properties.

Segmentation tasks involve identifying the precise location and shape of objects at pixel level. Traditional fully convolution networks (FCNs) allowed these end-to-end pixel wise predictions, but their course segmentation output often struggles to accurately identify boundaries. To overcome this, more advances models such as U-Net [4] and DeepLabv3+ [5] were developed which use atrous connections and skip connections and can be adapted to flower tasks. Although U-Net performs well with limited datasets, in this study, DeepLabv3+ with a ResNet-50 backbone is used to segment flower from their background. To explore its performance a separate daffodil dataset labelled for flower and background has been used.

In summary this study explores two key research questions:

1. Can a CNN trained from scratch achieve a high accuracy on a multi-class flower classification task?
2. Can DeepLabv3+ with ResNet-50 backbone provide accurate segmentation of daffodil images by separating flowers from complex backgrounds?

2. METHODS

The method used for the classification of flowers and segmentation of daffodils is outlined in this section. During the initial hyperparameter tuning process the data within the dataset was split into 70-10-20 where the training set was 70% of the data, 10% was validation and 20% of the images was used to test the model. This split facilitated the selection of optimal parameters. Once an acceptable result was obtained the model was retrained with an 80-20 split where 80% were training and 20% testing. This ensured that the final test set remained unseen throughout the tuning process to ensure an unbiased performance evaluation.

2.1. Classification

This section outlines the methodologies used to train a custom CNN for flower classification. The flower classification pipeline includes many key stages: data preparation, data augmentation, model design, training and lastly evaluation. Each step is described in detail in the following subsections.

The Oxford 17 flower dataset was used for classification which contains 1360 colour images distributed across 17 flower categories. Analysing the images showed that they

were captured under varying lighting conditions and different angles which resulted in different resolutions and scale. Therefore, the first step of preparing the data was resizing all the images to a fixed resolution of 256x256 pixels to ensure consistency and compatibility with the CNN.

MATLAB provides an automated labelling functionality called imageDatastore which loads and associates labels based on folder names. To use this function, the dataset had to be reorganised into subfolders corresponding to a flower class and the appropriate flower placed inside. This was achieved programmatically by looping through the dataset and associating each 80 set of images to a folder.

Augmentation was applied to the training set to artificially increase the data diversity to compensate for the small dataset. The data augmentation included random rotations with ±30, X and Y translations up to ±20 pixels, shear transformations ±20 in both directions, random scaling between 0.8 and 1.2 and horizontal flipping. These transformations were applied using MATLAB's imageDataAugmenter and augmentImageDatastore. This mitigated the possibility of overfitting caused by the small dataset and performing poor on unseen data. Applying extensive augmentation meant that during the hyperparameter tuning process the validation accuracy was above the train accuracy, showing that the model was performing well on unseen data.

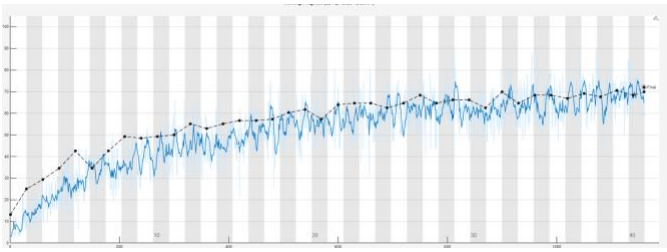


Figure 1: Validation/training accuracy graph

A custom convolutional neural network (CNN) was designed with five convolutional blocks to extract progressively complex spatial features. The first convolutional layer used 32 filters of size 5×5, selected to capture broader texture and colour gradients in early layers. The following layers used 64, 128, 256, and 512 filters with 3×3 kernels, allowing hierarchical feature abstraction with manageable computational cost. Smaller kernels in deeper layers improve the localisation of finer visual details, which is essential for distinguishing subtle floral structures.

Each convolutional layer was followed by batch normalisation, which standardises activations and accelerates convergence while improving generalisation [6]. ReLU activation functions were used throughout due to their simplicity and effectiveness in preventing missing local maxima.

A 2×2 max pooling layer with stride 2 was applied after each convolutional block to reduce spatial dimensions and computation while preserving dominant features. This also introduces a degree of translational invariance, useful in handling image variability.

The fully connected section consisted of 512 and 128 neurons, followed by the output layer with 17 neurons which corresponds to the number of flower classes. These layer sizes were chosen to balance capacity and overfitting risk. Dropout was applied at rates of 0.4 and 0.3, respectively, to regularise the network by preventing co-adaptation of neurons [7]. These rates were selected during hyperparameter tuning to ensure optimal generalisation on the small dataset.

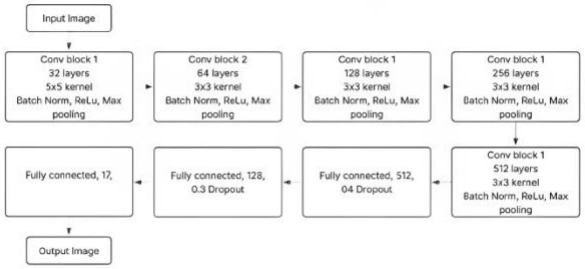


Figure 2: Diagram of the custom CNN architecture showing five convolutional blocks followed by three fully connected layers.

Model training was conducted using MATLAB's trainNetwork function. The optimiser selected for this task was Stochastic Gradient Descent with Momentum (SGDM). This was chosen due to its stability and effectiveness in training convolutional networks especially on smaller datasets compared to adaptive methods. A learning rate of 0.0002, determined during the 70-10-20 validation phase, allowed stable weight updates and minimised to risk of skipping over optimal minima. The batch size of 32 provided a good trade-off between computational efficiency and stability. The model was trained for 40 epochs which resulted in an average total training time of 15 minutes using a single GPU.

Once trained the model was evaluated using accuracy, per – class precision, recall, and F1 score. A confusion matrix was also created so misclassification across the 17 classes can be visualised. These metrics were computed using MATLAB'S evaluateClassificationModel and confusionmat built-in functions.

2.2. Segmentation

The segmentation task was conducted on a custom daffodil dataset with the objective of distinguishing flower pixels from non-flower pixels. All input images were resized to 256 x 256 pixels. Initially the ground truth masks contained multiple labels values representing different background categories. However, the goal of the segmentation was to identify flower regions, so all non-flower pixel labels were consolidated and remapped to 0 to represent the background.

This was implemented programmatically using a transformation function. Like classification, data augmentation is applied to the training images and corresponding label masks. This enhanced the model generalisation by simulating real world variations in flower appearance such as changes in orientation and scale.

The DeepLabv3+ pre-trained architecture was selected due to its proven performance in fine grained boundary aware segmentation tasks. This is particularly useful for flowers where petal edges and narrow stems require precise analysis. The model uses a ResNet-50 backbone for feature extraction, The encoder–decoder structure of this model allows for effective localisation of object boundaries, which is especially critical in flower segmentation as details like petal edges need to be preserved.

Training was conducted using MATLAB's TrainNetwork function with the Adam optimiser as it provides adaptive learning rates and faster convergence. The initial learning rate was set to 1×10^{-4} which helps to avoid divergence early. The model was trained for 20 epochs using a mini batch size of 4, which balances the GPU memory constraints. To prevent the model from learning from fixed patterns in the input order the data was shuffled each epoch. Using this training configuration, the model was trained for an average of 5 minutes.

The segmentation model was assessed using global accuracy, mean accuracy, intersection over union (IoU), and boundary F1 Score. A binary label map was used to compare the predicted mask with the ground truth and per class evaluations were also computed.

3. EVALUATION AND DISCUSSION

3.1. Classification Results

The performance of the CNN was evaluated using standard metrics such as overall accuracy, per class precision, recall, and F1-score. The model achieved an overall test accuracy of 73.90% with a mean precision of 73.90, recall of 75.06% and F1-score of 73.86 which showcases the model's ability to categorise unseen data. These results were averaged over multiple runs with random 80-20 splits to mitigate fluctuations caused by data shuffling and ensure robustness.

The confusion matrix shown in Figure 3 shows the number of correct and incorrect predictions across all 17 flower classes. High diagonal values show the model's strong performance in correctly identifying the class such as sunflower where all 16 of the images were correctly classified. The strong performance on this class suggests the model successfully captures distinguishing features as the sunflowers were strikingly unique compared to the other flowers where multiple features were similar. However certain classes such as Cowslip and Tulip were often misclassified which highlights the limitations in the models feature representation. These flowers share a similar yellow colour and petal arrangement as shown in figure 4, which likely contributed to the networks struggle to classify. The early layers of the CNN pick up on colour gradients which causes a confusion.

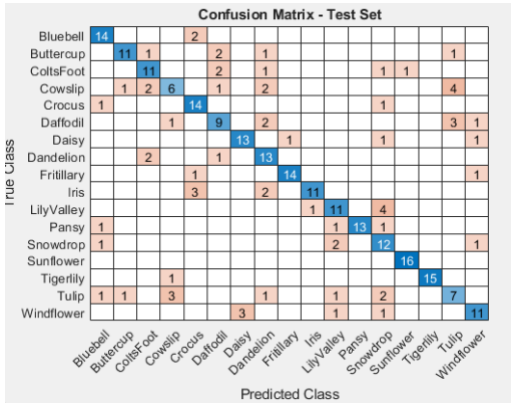


Figure 3 : Flower classification confusion matrix



Figure 4: Flower image of Cowslip on left and Tulip on right.

Table 1 presents a detailed per class evaluation metrics, where classes such as Cowslip, Buttercup and Coltsfoot achieved near perfect F1 scores. This indicates an excellent recognition accuracy. These classes have unique visual patterns which likely aided correct classification.

Class	Precision	Recall	F1Score
{'Daffodil' }	0.875	0.77778	0.82353
{'Snowdrop' }	0.6875	0.84615	0.75862
{'LilyValley'}	0.6875	0.6875	0.6875
{'Bluebell' }	0.375	0.54545	0.44444
{'Crocus' }	0.875	0.7	0.77778
{'Iris' }	0.5625	0.6	0.58065
{'Tigerlily' }	0.8125	0.8125	0.8125
{'Tulip' }	0.8125	0.59091	0.68421
{'Fritillary'}	0.875	0.93333	0.90323
{'Sunflower' }	0.6875	0.91667	0.78571
{'Daisy' }	0.6875	0.6875	0.6875
{'ColtsFoot' }	0.8125	1	0.89655
{'Dandelion' }	0.75	0.52174	0.61538
{'Cowslip' }	1	0.94118	0.9697
{'Buttercup' }	0.9375	1	0.96774
{'Windflower'}	0.4375	0.46667	0.45161
{'Pansy' }	0.6875	0.73333	0.70968

Table 1: Per class metrics table

The model's architecture and augmentation strategy contributed to the strong performance [8]. However, the absence of a pre-trained weight, meant the CNN required a

high epoch to converge which means a longer training time. Additionally, the limited dataset constrained the model's ability to capture complex intra-class variation. Future performance could be enhanced by employing a fine-tuned pre-trained model, such as ResNet or MobileNet, which have reported accuracies above 85% on the Oxford 17 dataset in prior studies [1], suggesting potential gains over the current 73.9%.

3.2. Segmentation Results

The segmentation performance of the DeepLabv3+ model with a ResNet-50 backbone was evaluated and achieved a global accuracy of 96.93% with a mean accuracy of accuracy of 97.25%, mean Intersection-over-Union (IoU) of 92.21%, and a mean Boundary F1 Score (BFScore) of 78% which shows the model performs accurate segmentation.

A closer analysis of the per class metrics shown in figure 5 reveals that the background class achieved a near perfect IoU score of 95% and a mean BF score of 87%. This result was expected due to the larger number of background pixels and their low variability. This class therefore benefits from consistent patterns across training samples, making it easier for the model to learn and generalise background pixels. In contrast the flower class, although having high accuracy and IoU, has a comparatively lower mean BF score of 78%. This suggest that, although the model correctly segments flower pixels, it struggles to precisely delineate flower boundaries, particularly where petal edges are thin, overlapping, or fade into the background [9]. This is consistent with challenges commonly faced in segmentation of organic structures.

	Accuracy	IoU	MeanBFScore
background	0.97527	0.96853	0.90571
flower	0.98066	0.91756	0.7842

Figure 5: Per class metrics table

The datasets class balance where background pixels greatly outnumber flower pixels, likely cause the model to bias towards background pixels. Despite data augmentation and binary label remapping, this imbalance may have reduced sensitivity to subtle flower features, particularly for small or partially occluded flowers [10].

Figure 6 shows the results of 6 tests images where the red region represents predicted flower segments overlaid onto the original image. The predictions demonstrate strong alignment with flower regions especially in images that are centred.

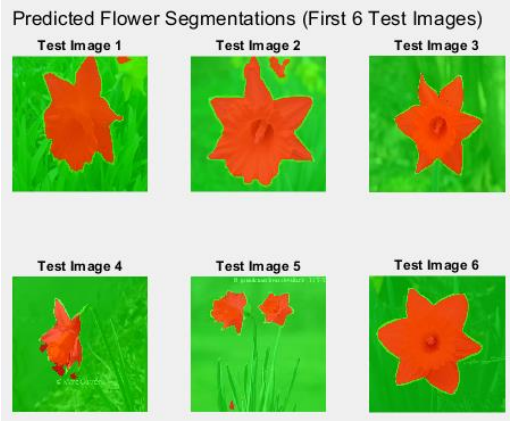


Figure 6: Example segmentation results overlayed onto original image.

The pixel-level confusion matrix in Figure 6 supports these findings. The model correctly predicted 215,495 flower pixels and 673,838 background pixels, with relatively few misclassifications overall. This reinforces the quantitative metrics and confirms that the model is highly effective in most scenarios but failed along thin petal boundaries.

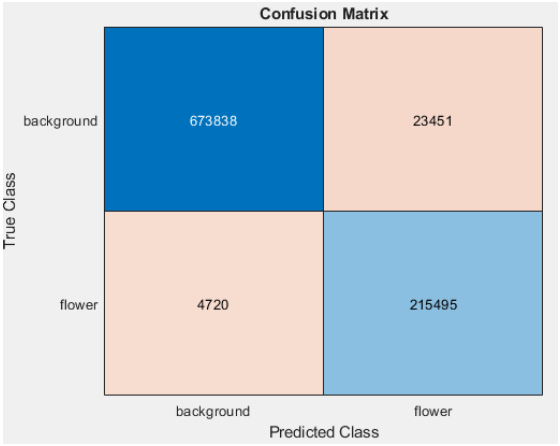


Figure 7: segmentation confusion matrix

5. CONCLUSION

This study evaluated two deep learning approaches for flower image analysis: a scratch-trained CNN for multi-class classification and a pre-trained DeepLabv3+ model for daffodil segmentation. The results supported the hypothesis that a custom CNN can perform competitively without transfer learning, while DeepLabv3+ offers strong segmentation accuracy. Despite some limitations in boundary precision, both models demonstrated effective generalisation. These findings highlight the potential of lightweight CNNs for constrained classification tasks and the continued value of transfer learning for precise segmentation. Future work may focus on refining boundary accuracy using edge-aware losses or attention-based architectures.

REFERENCES

[1] M.-E. Nilsback and A. Zisserman, "Automated Flower Classification over a Large Number of Classes," *2008 Sixth Indian Conference on Computer Vision, Graphics & Image Processing*,Dec. 2008,doi:https://doi.org/10.1109/icvgip.2008.47

[2] K. Simonyan and A. Zisserman, "VERY DEEP CONVOLUTIONAL NETWORKS FOR LARGE-SCALE IMAGE RECOGNITION," Apr. 2015. Available: https://arxiv.org/pdf/1409.1556

[3] K. He, X. Zhang, S. Ren, and J. Sun, "Deep Residual Learning for Image Recognition," arXiv.org, Dec. 10, 2015. https://arxiv.org/abs/1512.03385

[4] B. Bai, J. Zhang, Z. Song, P. Zhou, and A. Zhao, "Enhancing Sunflower Head Segmentation and Counting Using Improved UNet," Applied Engineering in Agriculture, vol. 41, no. 2, pp. 125–136, Jan. 2025, doi: https://doi.org/10.13031/aea.16163

[5] H. Peng et al., "Semantic Segmentation of Litchi Branches Using DeepLabV3+ Model," vol. 8, pp. 164546–164555, Jan. 2020, doi: https://doi.org/10.1109/access.2020.3021739

[6] S. Ioffe and C. Szegedy, "Batch Normalization: Accelerating Deep Network Training by Reducing Internal Covariate Shift," *arXiv.org*, 2015. https://arxiv.org/abs/1502.03167

[7] Nitish Srivastava, G. E. Hinton, A. Krizhevsky, and Ruslan Salakhutdinov, "Dropout: A Simple Way to Prevent Neural Networks from Overfitting," ResearchGate, Jun. 2014. https://www.researchgate.net/publication/286794765_Dropout_A_Simple_Way_to_Prevent_Neural_Networks_from_Overfitting

[8] L. Perez and J. Wang, "The Effectiveness of Data Augmentation in Image Classification using Deep Learning," *arXiv.org*, Dec. 13, 2017. https://arxiv.org/abs/1712.04621v1

[9] O. Ronneberger, P. Fischer, and T. Brox, "U-Net: Convolutional Networks for Biomedical Image Segmentation," *Lecture Notes in Computer Science*, vol. 9351, pp. 234–241, 2015, doi: https://doi.org/10.1007/978-3-319-24574-4_28

[10] L.-C. Chen, Y. Zhu, G. Papandreou, F. Schroff, and H. Adam, "Encoder-Decoder with Atrous Separable Convolution for Semantic Image Segmentation," arXiv.org, 2018. https://arxiv.org/abs/1802.02611